

Parallelization of Full-Field Reorientation in Microstructure Evolution

Chun-Mei Yang
314551115

Jyun Lai
314553031

Chi-Yi Lin
314581003

1 Introduction/motivation

Computational materials science has become a cornerstone of modern engineering, enabling the design and optimization of advanced materials. Among the key physical phenomena of interest is microstructure evolution—including recrystallization and grain growth—which determines the mechanical properties of interstitial-free (IF) steels and similar alloys [4].

The open-source framework **OMicroN** [3] models these processes using a cellular automaton (CA) approach, where local reorientation and growth are driven by curvature minimization and energetic balance. While accurate, this method is computationally expensive. Profiling reveals that the vast majority of runtime is spent in a single kernel function: `CalcReorientationRatesForFullFieldRexOrGG`, which iterates over tens of thousands of interface cells per timestep and performs intensive floating-point operations.

This bottleneck limits simulation scale and time resolution. In this project, we propose to parallelize this function using OpenMP, along with the color-coding visualization stage. Our goal is to improve throughput and scalability while maintaining numerical correctness and reproducibility.

2 Statement of the problem

2.1 The OMicroN Simulation Framework

OMicroN operates on a 2D or 3D cellular grid, where each cell stores state variables such as orientation, phase, and energy. In each timestep, the main loop calls the reorientation kernel to compute orientation change rates, then updates the microstructure accordingly. Visualization is performed by mapping orientations to colors, generating micrographs for analysis.

2.2 Computational Bottleneck

The function `CalcReorientationRatesForFullFieldRexOrGG` computes misorientations and energy changes across all interface cells, with complexity roughly $O(N_{icells} \times k^2)$, where N_{icells} is the number of interface cells and k the neighbor count. Profiling shows this kernel consumes over 80% of total runtime, largely due to:

- Non-contiguous memory access (AoS layout) causing poor cache reuse.
- Heavy per-cell computation (multiple trigonometric operations).
- Serial traversal of `std::set<int>` structures (no random-access iterators).

Addressing these inefficiencies through OpenMP parallelization and data layout optimization is expected to yield substantial speedup.

3 Proposed Approaches

Our proposed approach applies OpenMP-based shared-memory parallelism to both the reorientation kernel and the visualization stage. The overall design is summarized in Figure 1.

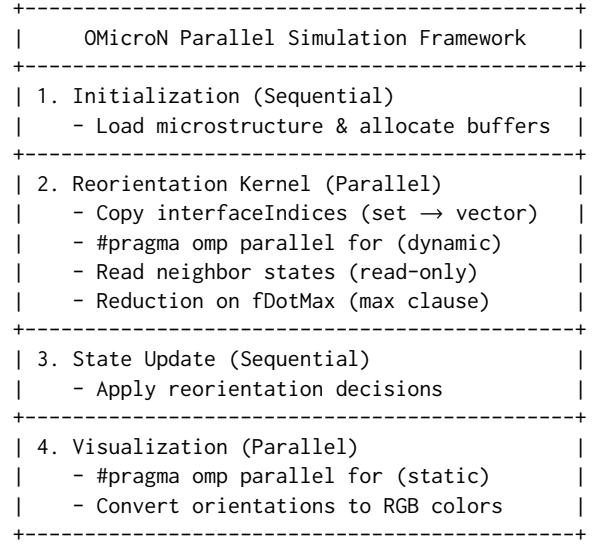


Figure 1: Proposed parallel system architecture for OMicroN.

3.1 Reorientation Kernel Parallelization

The kernel iterates over a `std::set<int>` of interface cell indices. Since OpenMP requires random-access iterators, we will convert this set into a `std::vector<int>` prior to entering the parallel region. The copying overhead is negligible compared to the $O(N * k^2)$ computational workload, as demonstrated by the following analysis.

The copying overhead is negligible compared to the $O(N \times k^2)$ computational workload. To demonstrate this, we can perform a brief complexity analysis. Let N_{icells} be the number of interface cells and k be the number of neighbors for each cell. The cost of converting the `std::set` to a `std::vector`, C_{conv} , is linear with the number of elements, thus $C_{conv} = O(N_{icells})$. In contrast, the core computation cost, C_{comp} , is approximately $O(N_{icells} \times k^2)$.

Consequently, the ratio of these two costs can be expressed as:

$$\frac{C_{conv}}{C_{comp}} = \frac{O(N_{icells})}{O(N_{icells} \times k^2)} = O\left(\frac{1}{k^2}\right)$$

This cost ratio of $O(1/k^2)$ supports our hypothesis that the conversion overhead is asymptotically smaller and will be dominated by the computational workload. For instance, in a 2D simulation with

8 neighbors, the computation is theoretically over $8^2 = 64$ times more expensive. We will empirically validate this during Week 7 by profiling the simulation to confirm the overhead is insignificant in practice.

Based on this analysis, we will proceed with the conversion and apply the following OpenMP directives:

- `#pragma omp parallel for schedule(dynamic)` to distribute variable workloads.
- `reduction(max: fDotMax)` to safely compute the global maximum.
- Data layout conversion from AoS to SoA to improve cache locality.

3.2 Visualization Parallelization

The `GetColorCoding` function maps crystal orientations to RGB color values. This step is embarrassingly parallel. We apply `#pragma omp parallel for schedule(static)` for even work distribution and minimal synchronization.

4 Language Selection

We choose **OpenMP with C++** for the following reasons:

- OMicroN is implemented in C++, and OpenMP integrates seamlessly with it.
- OpenMP offers a shared-memory parallel model that matches the per-voxel computation pattern.
- It requires minimal code modification for rapid prototyping.
- Provides straightforward tools for performance tuning (e.g., `num_threads`, `scheduling`).

5 Related Work

Several studies discuss the computational aspects of recrystallization and texture evolution:

- Traka et al. [4] described the topological mechanisms of recrystallization in IF steel and introduced the algorithm underlying OMicroN.
- Després et al. [1] proposed mean-field approaches for orientation spread modeling.
- Zhang et al. [5] and Zhao et al. [6] demonstrated parallel Monte Carlo and phase-field simulations of microstructure evolution.
- Parallel programming methods for scientific kernels are widely studied in OpenMP tutorials [2].

Our work focuses on integrating OpenMP parallelization within an existing full-field model, improving scalability and efficiency.

6 Statement of Expected Results

- Achieve significant speedup (target: $>5\times$ on 8 threads) in the reorientation rate calculation.
- Reduce total runtime of OMicroN’s full-field evolution step.
- Maintain identical physical outputs to serial results (within floating-point tolerance).
- Provide detailed performance profiling: speedup, parallel efficiency, cache miss trends.
- Deliver a documented and reproducible OpenMP implementation.

7 A Timetable

Table 1: Project schedule from Week 6 (proposal) to Week 13 (final presentation).

Week	Tasks and Milestones
6	Submit the project proposal. Define objectives, scope, and review related work.
7	Set up OMicroN environment. Run and profile baseline simulations. Identify major bottlenecks in <code>CalcReorientationRatesForFullFieldRexOrGG</code> .
8	Analyze data structures and algorithmic dependencies. Design OpenMP parallelization plan including tiling, buffering, and scheduling.
9	Implement first version of OpenMP-parallelized reorientation kernel. Validate correctness against serial results.
10	Performance tuning I: Test different scheduling strategies (static/dynamic/guided) and measure cache performance.
11	Parallelize color-mapping (visualization) module. Integrate full workflow and prepare preliminary report.
12	Performance tuning II: Refine memory locality and load balancing. Finalize experiments and complete report.
13	Final project presentation. Deliver slides, final report, and source code submission.

8 References

References

- [1] Arthur Després, Michael Greenwood, and Cameron Sinclair. 2020. A mean-field model of static recrystallization considering orientation spreads and their time-evolution. *arXiv preprint arXiv:2005.07230* (2020). <https://arxiv.org/abs/2005.07230>
- [2] Lawrence Livermore National Laboratory. 2023. OpenMP Tutorial: Shared Memory Parallel Programming. <https://hpc-tutorials.llnl.gov/openmp/>. Accessed 2025-10-10.
- [3] Konstantinos Traka. 2023. OMicroN: Open Microstructure Nucleation and Recrystallization Simulation Framework. <https://github.com/k-traka/OMicroN>. GitHub repository.
- [4] Konstantinos Traka, K. Sedighiani, C. Bos, J. Galan Lopez, Katja Angenendt, D. Raabe, and J. Sietsma. 2021. Topological aspects responsible for recrystallization evolution in an IF-steel sheet – Investigation with cellular-automaton simulations. *Computational Materials Science* 198 (2021), 110693. doi:10.1016/j.commatsci.2021.110693
- [5] Liangzhe Zhang, Timothy Bartel, and Mark T. Lusk. 2010. Parallelized Hybrid Monte Carlo Simulation of Stress-Induced Texture Evolution. *arXiv preprint arXiv:1009.0457* (2010). <https://arxiv.org/abs/1009.0457>
- [6] Peng Zhao, T.S.E. Low, Y. Wang, and S. Niezgoda. 2015. An integrated full-field model of concurrent plastic deformation and microstructure evolution: Application to 3D simulation of dynamic recrystallization. *arXiv preprint arXiv:1509.04953* (2015). <https://arxiv.org/abs/1509.04953>